

项目报告

Web 前端开发

个人博客系统

专 业： 软件工程
班 级： 24 软工 1 班
学 号： xxxxxx
姓 名： xxx
指导教师： 冯新元

2026 年 6 月 20 日

目录

1	设计概述	4
1.1	设计目的	4
1.2	需求分析	4
1.2.1	功能需求	4
1.2.2	样式需求	4
1.2.3	页面流程图	5
2	技术简介	5
2.1	开发工具介绍	5
2.1.1	Visual Studio Code	5
2.1.2	Node.js	6
2.1.3	Vue CLI	6
2.1.4	Maven	6
2.1.5	MySQL	6
2.2	关键技术介绍	6
2.2.1	Vue 3 框架	6
2.2.2	Vue Router 4	6
2.2.3	Pinia 状态管理	6
2.2.4	Element Plus	7
2.2.5	Axios HTTP 客户端	7
2.2.6	ECharts 数据可视化	7
2.2.7	Markdown 编辑器	7
2.2.8	Sass 预处理器	7
3	设计步骤	7
3.1	项目架构设计	7
3.1.1	项目目录结构	7
3.1.2	路由配置	8
3.1.3	状态管理设计	9
3.2	头部与导航栏设计	9
3.2.1	头部布局设计	9
3.2.2	导航菜单实现	9
3.2.3	滚动效果实现	10
3.3	首页设计	11
3.3.1	英雄区域 (Hero Section)	11
3.3.2	粒子动画 CSS 实现	12
3.3.3	文章列表展示	13

3.3.4	文章卡片组件	14
3.4	文章浏览页设计	15
3.4.1	页面布局	15
3.4.2	Markdown 渲染	15
3.4.3	目录导航组件	16
3.4.4	评论系统设计	17
3.5	用户认证设计	18
3.5.1	登录页面设计	18
3.5.2	Pinia 用户状态管理	20
3.5.3	Axios 请求拦截器	21
3.6	后台管理系统设计	22
3.6.1	后台布局组件	22
3.6.2	仪表盘设计	23
3.6.3	文章管理	26
3.7	响应式布局设计	27
3.7.1	Flexbox 弹性布局	27
3.7.2	CSS Grid 网格布局	27
3.7.3	Element Plus 响应式组件	28
3.8	底部设计	28
3.8.1	底部布局	28
4	总体页面效果	29
4.1	前台页面效果	29
4.1.1	首页效果	29
4.1.2	文章列表页效果	29
4.1.3	文章详情页效果	29
4.1.4	分类/标签页效果	29
4.1.5	用户认证页面效果	30
4.1.6	个人中心页面效果	30
4.2	后台管理页面效果	30
4.2.1	仪表盘效果	30
4.2.2	文章管理效果	30
4.2.3	文章编辑页面效果	30
4.2.4	用户/评论管理效果	30
5	心得体会	30
5.1	技术能力提升	31
5.2	工程化意识培养	31
5.3	UI/UX 设计感悟	31
5.4	问题解决能力	32

5.5 总结与展望	32
---------------------	----

1 设计概述

1.1 设计目的

本项目旨在设计并实现一个功能完备、界面美观的个人博客系统。通过该项目的开发，深入实践 Vue3 前端框架的核心技术，包括组件化开发、响应式数据绑定、路由管理、状态管理等，同时掌握前后端分离架构下的前端开发流程。

具体设计目的包括：全面检验对 Vue3 课程中基础理论知识的理解与掌握情况，如 Vue3 的响应式原理、Composition API、组件化开发、状态管理（Pinia）、路由导航（Vue Router）等核心知识点；培养将 Vue3 知识与其他前端技术（HTML、CSS、JavaScript、Axios 等）综合运用能力；通过设计并开发具有实际功能的前端项目，提升综合应用与创新能力；深入理解前后端分离架构的设计思想，掌握与后端 RESTful API 接口的联调方法。

本博客系统采用 Vue3 作为前端框架，Element Plus 作为 UI 组件库，实现了用户认证、文章管理、分类标签、评论互动、友情链接、后台管理等核心功能。前端通过 Axios 与 Spring Boot 后端进行数据交互，采用 JWT Token 实现身份认证，Markdown 编辑器支持富文本文章发布，ECharts 实现数据可视化展示。

1.2 需求分析

根据项目要求，本博客系统需要满足以下功能需求和样式需求：

1.2.1 功能需求

系统包含前台展示功能和后台管理功能两大模块。前台展示功能包括：博客首页展示（最新文章列表、热门推荐、置顶文章）、文章浏览详情（支持 Markdown 渲染、点赞、收藏）、评论互动系统（支持多级评论回复）、分类与标签筛选、全文搜索、友情链接展示、关于我页面。

用户认证功能包括：用户注册（用户名、密码、邮箱）、用户登录（JWT Token 认证）、忘记密码重置。后台管理功能包括：仪表盘数据统计（ECharts 可视化图表）、文章管理（发布、编辑、删除、Markdown 编辑器）、分类管理、标签管理、评论审核、用户管理、系统设置。

1.2.2 样式需求

页面设计需满足以下样式需求：包含 Logo 和主题展示区；导航栏带下拉列表（二级菜单）；搜索栏支持关键词搜索；展示区包含文本、图像、列表、表格、表单、控件等多媒体元素；底部包含友情链接、联系方式、版权信息。页面布局采用响应式设计，适配不同屏幕尺寸的设备。

整体配色以深色为主调，搭配亮蓝色作为强调色，营造科技感和专业感。采用卡片式布局，圆角设计，阴影效果增强层次感。

1.2.3 页面流程图

系统的页面流程如图 1 所示。用户访问博客首页后，可通过导航栏访问文章列表、分类、标签、关于我等页面。已登录用户可发表评论、点赞、收藏文章，并进入个人中心管理资料。管理员可进入后台管理系统，进行文章发布、评论审核、用户管理等操作。

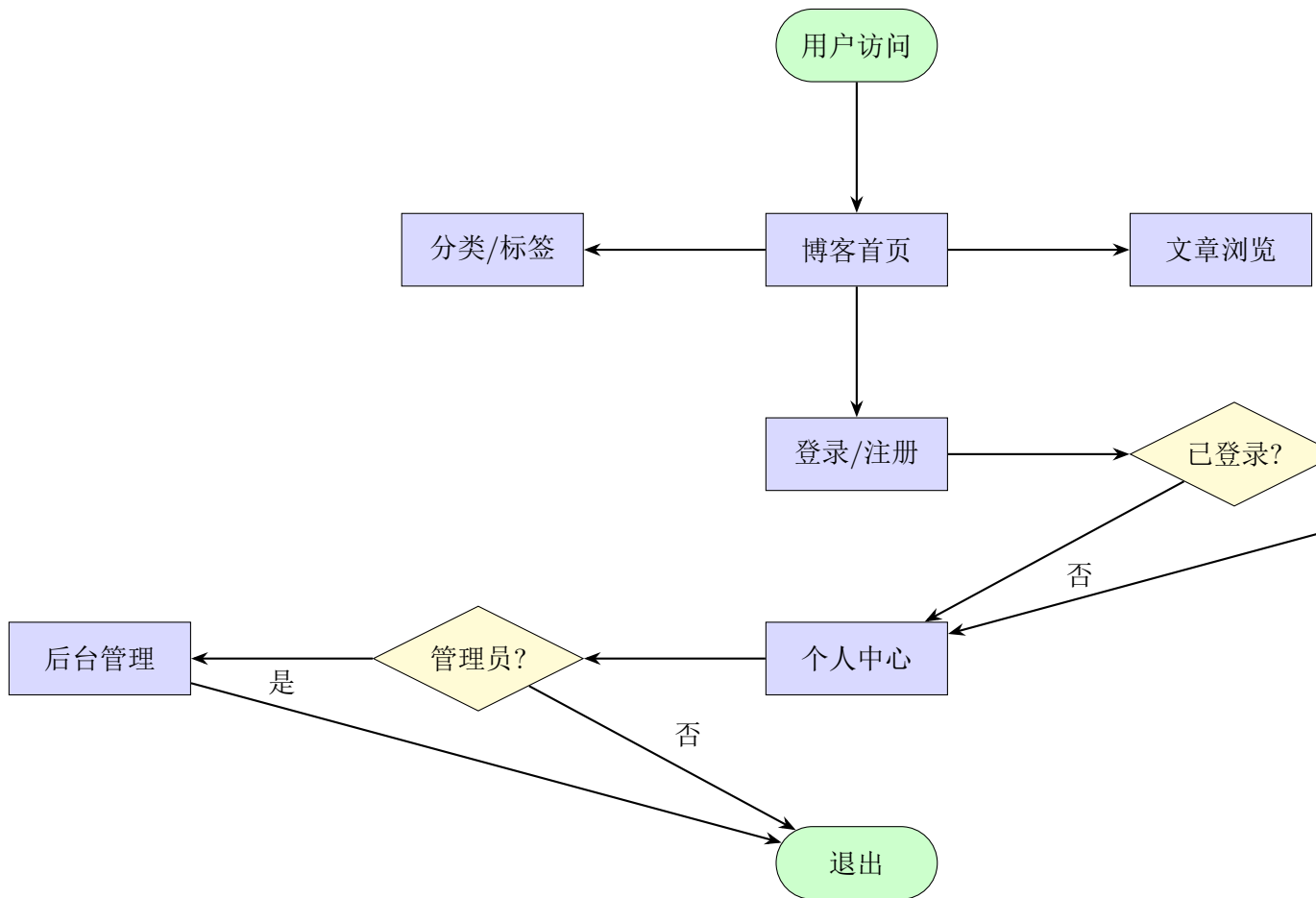


图 1: 系统页面流程图

2 技术简介

2.1 开发工具介绍

2.1.1 Visual Studio Code

Visual Studio Code（简称 VS Code）是微软开发的一款轻量级但功能强大的源代码编辑器。它支持多种编程语言，拥有丰富的插件生态系统。在本项目中，VS Code 作为主要的前端开发工具，配合 Vetur 插件提供 Vue 单文件组件的语法高亮、智能提示、代码格式化等功能，极大地提升了开发效率。

2.1.2 Node.js

Node.js 是一个基于 Chrome V8 引擎的 JavaScript 运行时环境。本项目使用 Node.js 22+ 版本，用于运行前端开发服务器、构建生产环境代码以及管理项目依赖。npm (Node Package Manager) 作为 Node.js 的包管理工具，用于安装 Vue CLI、Element Plus 等前端依赖库。

2.1.3 Vue CLI

Vue CLI 是 Vue.js 官方提供的脚手架工具，用于快速搭建 Vue 项目。本项目使用 @vue/cli 5.0 版本创建项目骨架，内置了 Babel 转译器、ESLint 代码检查、开发服务器热更新等功能。

2.1.4 Maven

Maven 是 Java 项目的构建和依赖管理工具。本项目的 Spring Boot 后端使用 Maven 3.9+ 进行构建管理。

2.1.5 MySQL

MySQL 8.0+ 是本项目使用的开源关系型数据库管理系统，用于存储用户信息、文章内容、分类标签、评论数据等核心数据。

2.2 关键技术介绍

2.2.1 Vue 3 框架

Vue 3 是渐进式 JavaScript 框架 Vue.js 的第三个主要版本。相比 Vue 2, Vue 3 引入了多项重要改进：采用 Proxy 实现更强大的响应式系统；Composition API 提供了更灵活的逻辑复用方式；性能优化使得渲染速度提升 1.3-2 倍；更完善的 TypeScript 支持。本项目采用 Vue 3.2 版本，使用 Composition API 组织组件逻辑。

2.2.2 Vue Router 4

Vue Router 是 Vue.js 官方的路由管理库。Vue Router 4 专为 Vue 3 设计，支持组合式 API。本项目使用 Vue Router 实现前端页面的路由导航，包括：声明式路由配置、嵌套路由、动态路由、路由守卫、路由懒加载。

2.2.3 Pinia 状态管理

Pinia 是 Vue 3 官方推荐的状态管理库，作为 Vuex 的继任者。Pinia 具有更简洁的 API 设计，更好的 TypeScript 支持，更轻量的体积。本项目使用 Pinia 3 管理全局状态。

2.2.4 Element Plus

Element Plus 是基于 Vue 3 的组件库，是 Element UI 的升级版。提供了丰富的 UI 组件，包括按钮、表单、表格、对话框、菜单、分页等。本项目使用 Element Plus 2.9 版本作为 UI 基础。

2.2.5 Axios HTTP 客户端

Axios 是一个基于 Promise 的 HTTP 客户端，用于浏览器和 Node.js 环境。本项目使用 Axios 1.7 版本与后端 RESTful API 进行数据交互。

2.2.6 ECharts 数据可视化

ECharts 是百度开源的数据可视化库。本项目在后台仪表盘使用 ECharts 5.6 展示博客核心数据统计。

2.2.7 Markdown 编辑器

本项目集成了 md-editor-v3 和 mavon-editor 两款 Markdown 编辑器，分别用于文章展示和文章编辑。

2.2.8 Sass 预处理器

Sass 是一种 CSS 预处理器，提供变量、嵌套、混合、继承等功能。本项目使用 Sass 编写组件样式。

3 设计步骤

3.1 项目架构设计

本项目采用前后端分离架构，前端基于 Vue 3 框架，后端基于 Spring Boot 框架。

3.1.1 项目目录结构

前端项目采用模块化的目录组织方式，主要目录包括：

- **src/api/** – 封装 Axios 请求，按模块组织 API 接口
- **src/assets/** – 存放静态资源文件（图片、字体、全局样式）
- **src/components/** – 可复用的公共组件
- **src/layouts/** – 页面布局组件（前台/后台）
- **src/router/** – Vue Router 路由配置

- **src/store/** – Pinia 状态管理
- **src/styles/** – 全局样式文件
- **src/utils/** – 工具函数
- **src/views/** – 页面视图组件，按功能模块分组

3.1.2 路由配置

路由配置采用模块化设计，将前台路由、后台路由分别定义。关键代码如下：

Listing 1: 路由配置代码

```
1 // router/index.js
2 import { createRouter, createWebHistory } from 'vue-router'
3 import { useUserStore } from '@store/user'
4 import BackendLayout from '@layouts/BackendLayout.vue'
5
6 // 后台路由
7 export const backendRoutes = [
8   {
9     path: '/back',
10    component: BackendLayout,
11    redirect: '/back/dashboard',
12    children: [
13      {
14        path: 'dashboard',
15        name: 'Dashboard',
16        component: () =>
17          import('@views/backend/Dashboard.vue'),
18        meta: { title: '首页', icon: 'HomeFilled' }
19      },
20      {
21        path: 'article',
22        name: 'ArticleManagement',
23        component: () =>
24          import('@views/backend/article/index.vue'),
25        meta: { title: '文章管理', icon: 'Document' }
26      }
27    ]
28  }
29 ]
```

```
29 // 路由守卫
30 router.beforeEach((to, from, next) => {
31   const userStore = useUserStore()
32   if (to.path.startsWith('/back') && !userStore.token) {
33     next('/login')
34   } else {
35     next()
36   }
37 })
```

3.1.3 状态管理设计

使用 Pinia 进行全局状态管理，定义了多个 Store 模块：userStore 管理用户登录状态；appStore 管理应用级状态；articleStore 管理文章相关状态。

3.2 头部与导航栏设计

头部导航栏是博客系统的重要组成部分，包含 Logo、主导航菜单、搜索栏、用户操作入口等元素。

3.2.1 头部布局设计

头部采用固定定位（position: fixed）始终显示在页面顶部，高度为 60px，背景色为半透明深色配合 backdrop-filter 毛玻璃效果。

3.2.2 导航菜单实现

导航菜单使用 Element Plus 的 el-menu 组件，支持二级下拉菜单。菜单项包括：首页、文章（下拉展开分类列表）、分类、标签、友情链接、关于。当前路由对应的菜单项高亮显示。

Listing 2: 头部导航组件代码

```
1 <template>
2   <header class="app-header" :class="{ scrolled: isScrolled }">
3     <div class="header-content">
4       <router-link to="/" class="logo">
5         
6         <span class="site-name">个人博客</span>
7       </router-link>
8       <el-menu :default-active="activeIndex" mode="horizontal"
          router
```

```
9         background-color="transparent" text-color="#fff"
10         active-text-color="#4facfe">
11         <el-menu-item index="/">
12             <el-icon><HomeFilled /></el-icon> 首页
13         </el-menu-item>
14         <el-sub-menu index="/article">
15             <template #title>
16                 <el-icon><Document /></el-icon> 文章
17             </template>
18             <el-menu-item v-for="cat in categories" :key="cat.id"
19                 :index="'/category/${cat.id}'">{{ cat.name
20                 }}</el-menu-item>
21         </el-sub-menu>
22     </el-menu>
23     <div class="search-box">
24         <el-input v-model="searchKeyword"
25             placeholder="搜索文章..."
26             @keyup.enter="handleSearch" />
27     </div>
28 </div>
29 </header>
30 </template>
```

3.2.3 滚动效果实现

通过监听窗口滚动事件，当滚动距离超过 50px 时为头部添加 scrolled 类名，改变背景透明度。使用 lodash 的 throttle 函数进行节流处理。

Listing 3: 滚动监听代码

```
1 // 滚动监听
2 import { ref, onMounted, onUnmounted } from 'vue'
3 import { throttle } from 'lodash'
4
5 const isScrolled = ref(false)
6 const handleScroll = throttle(() => {
7     isScrolled.value = window.scrollY > 50
8 }, 100)
9
10 onMounted(() => {
11     window.addEventListener('scroll', handleScroll)
12 })
```

```
13 onUnmounted(() => {  
14   window.removeEventListener('scroll', handleScroll)  
15 })
```

3.3 首页设计

首页是博客系统的门面，需要展示博客的核心内容并给用户留下良好的第一印象。

3.3.1 英雄区域 (Hero Section)

页面顶部为英雄区域，包含全屏的 3D 视差背景动画和欢迎文字。背景使用 CSS 动画创建 50 个浮动粒子效果，营造科技感。欢迎文字使用打字机效果逐字显示博客名称和副标题。右侧使用 Element Plus 的 el-carousel 轮播组件展示推荐文章封面图。底部使用 SVG 绘制三层波浪动画，实现平滑的过渡效果。

Listing 4: 英雄区域代码

```
1 <template>  
2   <div class="home-container">  
3     <div class="parallax-background">  
4       <div class="particle" v-for="n in 50" :key="n"></div>  
5     </div>  
6     <div class="hero-section">  
7       <div class="welcome-text">  
8         <h1 class="typewriter">{{ displayText }}</h1>  
9         <p class="subtitle">{{ subtitle }}</p>  
10      </div>  
11      <el-carousel height="450px" indicator-position="none"  
12        arrow="never" class="hero-carousel">  
13        <el-carousel-item v-for="item in recommendArticles"  
14          :key="item.id">  
15            
16        </el-carousel-item>  
17      </el-carousel>  
18    </div>  
19 </template>  
20  
21 <script setup>  
22 import { ref, onMounted } from 'vue'
```

```
23 const displayText = ref('')
24 const fullText = '欢迎来到我的个人博客'
25
26 onMounted(() => {
27   let index = 0
28   const timer = setInterval(() => {
29     if (index < fullText.length) {
30       displayText.value += fullText[index]
31       index++
32     } else {
33       clearInterval(timer)
34     }
35   }, 150)
36 })
37 </script>
```

3.3.2 粒子动画 CSS 实现

粒子动画使用纯 CSS 实现，通过为每个粒子设置随机的位置、大小、动画延迟和持续时间，营造星空般的视觉效果。

Listing 5: 粒子动画 CSS 代码

```
1  /* 粒子动画样式 */
2  .parallax-background {
3    position: fixed;
4    top: 0; left: 0;
5    width: 100%; height: 100%;
6    background: linear-gradient(135deg, #1a1a2e, #16213e);
7    z-index: -1;
8  }
9
10 .particle {
11   position: absolute;
12   width: 2px; height: 2px;
13   background: rgba(255,255,255,0.5);
14   border-radius: 50%;
15   animation: float linear infinite;
16 }
17
18 @keyframes float {
19   0% { transform: translateY(0) translateX(0); opacity: 0; }
```

```
20 10% { opacity: 1; }
21 90% { opacity: 1; }
22 100% { transform: translateY(-100vh) translateX(50px);
      opacity: 0; }
23 }
```

3.3.3 文章列表展示

首页主体区域展示最新文章列表，采用卡片式布局。每篇文章卡片包含封面图、标题、摘要、作者、发布时间、浏览量、分类标签等信息。使用 CSS Grid 布局实现自适应的列数。

Listing 6: 文章列表组件代码

```
1 <template>
2   <div class="article-section">
3     <h2 class="section-title">最新文章</h2>
4     <div class="article-grid">
5       <article-card
6         v-for="article in articleList"
7         :key="article.id"
8         :article="article"
9         @click="$router.push(`/article/${article.id}`)"
10      />
11    </div>
12    <el-pagination
13      v-model:current-page="pageNum"
14      v-model:page-size="pageSize"
15      :total="total"
16      :page-sizes="[9, 12, 15, 18]"
17      layout="total, sizes, prev, pager, next, jumper"
18    />
19  </div>
20 </template>
21
22 <script setup>
23 import { ref, onMounted } from 'vue'
24 import { getArticleList } from '@api/article'
25
26 const articleList = ref([])
27 const pageNum = ref(1)
```

```
28 const pageSize = ref(9)
29 const total = ref(0)
30
31 const fetchArticles = async () => {
32   const res = await getArticleList({
33     pageNum: pageNum.value,
34     pageSize: pageSize.value
35   })
36   articleList.value = res.data.list
37   total.value = res.data.total
38 }
39 onMounted(fetchArticles)
40 </script>
```

3.3.4 文章卡片组件

ArticleCard 是一个可复用的组件，接收 article 对象作为 props，展示文章的缩略信息。卡片采用圆角矩形设计，图片区域占高度的 60%，信息区域占 40%。

Listing 7: 文章卡片组件代码

```
1 <template>
2   <div class="article-card" :class="{ featured: article.isTop
3     }">
4     <div class="card-image">
5       
7       <span v-if="article.isTop" class="badge top">置顶</span>
8       <span v-if="article.isRecommend" class="badge
9         recommend">推荐</span>
10    </div>
11    <div class="card-body">
12      <h3 class="card-title">{{ article.title }}</h3>
13      <p class="card-summary">{{ article.summary }}</p>
14      <div class="card-meta">
15        <span><el-icon><User /></el-icon>{{ article.authorName
16          }}</span>
17        <span><el-icon><Clock /></el-icon>{{
18          formatDate(article.createTime) }}</span>
19        <span><el-icon><View /></el-icon>{{ article.viewCount
20          }}</span>
21      </div>
22    </div>
```

```
16     <div class="card-tags">
17       <el-tag v-for="tag in article.tagList" :key="tag.id"
18         size="small" effect="plain">{{ tag.name }}</el-tag>
19     </div>
20   </div>
21 </div>
22 </template>
```

3.4 文章浏览页设计

文章详情页是博客最核心的页面，需要优雅地展示文章内容并提供良好的阅读体验。

3.4.1 页面布局

文章详情页采用左右两栏布局。左侧为主内容区（占 8/12 宽度），包含文章标题、元信息、正文内容、标签、点赞收藏按钮、评论区。右侧为侧边栏（占 4/12 宽度），包含作者信息卡片、目录导航、相关文章推荐。

3.4.2 Markdown 渲染

文章内容使用 Markdown 格式存储，前台展示时使用 md-editor-v3 的预览组件进行渲染。支持代码高亮、表格、图片、数学公式等 Markdown 语法。

Listing 8: 文章详情页代码

```
1 <template>
2   <div class="article-detail">
3     <div class="content-wrapper">
4       <main class="main-content">
5         <div class="article-header">
6           <h1 class="article-title">{{ article.title }}</h1>
7           <div class="article-meta">
8             
10            <span>{{ article.authorName }}</span>
11            <el-icon><Clock /></el-icon>
12            <span>{{ formatDate(article.createTime) }}</span>
13            <el-icon><View /></el-icon>
14            <span>{{ article.viewCount }}次浏览</span>
15          </div>
16        </div>
17      </div>
18    </div>
19  </div>
```



```

16     <md-preview :modelValue="article.content" preview-only
17       />
18     <div class="article-tags">
19       <el-tag v-for="tag in article.tagList" :key="tag.id">
20         {{ tag.name }}</el-tag>
21     </div>
22     <div class="article-actions">
23       <el-button :type="isLiked ? 'danger' : 'default'"
24         @click="handleLike">
25         <el-icon><Pointer /></el-icon>{{ article.likeCount
26           }} 点赞
27       </el-button>
28       <el-button :type="isCollected ? 'primary' :
29         'default'"
30         @click="handleCollect">
31         <el-icon><Star /></el-icon>{{ article.collectCount
32           }} 收藏
33       </el-button>
34     </div>
35     <comment-list :article-id="articleId"
36       :comments="comments" />
37   </main>
38   <aside class="sidebar">
39     <author-card :author="article.author" />
40     <toc-nav :content="article.content" />
41   </aside>
42 </div>
43 </template>

```

3.4.3 目录导航组件

目录导航组件（TOC）自动解析文章中的标题标签（h1-h4），生成可点击的目录树。使用 Intersection Observer API 监听标题元素的可见性，高亮当前阅读的章节。

Listing 9: 目录导航组件代码

```

1 // TocNav.vue - 目录导航
2 import { ref, onMounted } from 'vue'
3 import MarkdownIt from 'markdown-it'
4
5 const tocItems = ref([])

```

```
6  const activeId = ref('')
7
8  const generateToc = (markdown) => {
9    const md = new MarkdownIt()
10   const tokens = md.parse(markdown, {})
11   const headings = []
12   tokens.forEach((token, i) => {
13     if (token.type === 'heading_open') {
14       const level = parseInt(token.tag[1])
15       const content = tokens[i + 1].content
16       headings.push({ level, content, id: `h-${i}` })
17     }
18   })
19   tocItems.value = headings
20 }
21
22 onMounted(() => {
23   const observer = new IntersectionObserver((entries) => {
24     entries.forEach(entry => {
25       if (entry.isIntersecting) {
26         activeId.value = entry.target.id
27       }
28     })
29   }, { rootMargin: '-100px 0px -60% 0px' })
30   document.querySelectorAll('h1, h2, h3, h4').forEach(h => {
31     observer.observe(h)
32   })
33 })
```

3.4.4 评论系统设计

评论系统支持多级嵌套回复。数据库采用自关联表结构（parent_id 字段），前端通过递归组件渲染嵌套评论树。

Listing 10: 递归评论组件代码

```
1  <template>
2    <div class="comment-section">
3      <h3>评论 ({{ total }})</h3>
4      <div v-if="isLoggedIn" class="comment-input">
5        <el-input v-model="commentContent" type="textarea"
6          :rows="3" placeholder="写下你的评论..." />
```

```
7       <el-button type="primary"
8         @click="submitComment">发表评论</el-button>
9     </div>
10    <div class="comment-list">
11      <comment-item v-for="comment in commentTree"
12        :key="comment.id"
13        :comment="comment" @reply="handleReply" />
14    </div>
15  </div>
16 </template>
17
18 <script setup>
19 import { ref, computed } from 'vue'
20 const props = defineProps({ comments: Array })
21 const commentTree = computed(() => {
22   const map = {}
23   const roots = []
24   props.comments.forEach(c => { map[c.id] = { ...c, children:
25     [] } })
26   props.comments.forEach(c => {
27     if (c.parentId && map[c.parentId]) {
28       map[c.parentId].children.push(map[c.id])
29     } else { roots.push(map[c.id]) }
30   })
31   return roots
32 })
33 </script>
```

3.5 用户认证设计

用户认证是博客系统的基础功能，包括注册、登录、登出、权限控制等功能。

3.5.1 登录页面设计

登录页面采用居中卡片布局，背景为动态粒子效果。卡片内包含用户名输入框、密码输入框、验证码输入框、记住我选项和登录按钮。表单验证使用 Element Plus 的 rules 配置，在提交前进行前端校验。

Listing 11: 登录页面代码

```
1 <template>
2   <div class="auth-page">
```

```
3      <div class="auth-card">
4        <h2>{{ isLogin ? '用户登录' : '注册账号' }}</h2>
5        <el-form :model="form" :rules="rules" ref="formRef"
6          @keyup.enter="handleSubmit">
7          <el-form-item prop="username">
8            <el-input v-model="form.username"
9              placeholder="用户名"
10              :prefix-icon="User" size="large" />
11          </el-form-item>
12          <el-form-item prop="password">
13            <el-input v-model="form.password" type="password"
14              placeholder="密码" :prefix-icon="Lock"
15              size="large" show-password />
16          </el-form-item>
17          <el-form-item prop="captcha">
18            <div class="captcha-row">
19              <el-input v-model="form.captcha"
20                placeholder="验证码"
21                size="large" style="flex: 1" />
22              
24            </div>
25          </el-form-item>
26          <el-button type="primary" size="large"
27            :loading="loading"
28            @click="handleSubmit">{{ isLogin ? '登录' : '注册' }}</el-button>
29        </el-form>
30      </div>
31    </div>
32  </template>
33
34  <script setup>
35  import { ref, reactive } from 'vue'
36  import { useRouter } from 'vue-router'
37  import { useUserStore } from '@store/user'
38
39  const router = useRouter()
40  const userStore = useUserStore()
41  const form = reactive({
42    username: '', password: '', captcha: '', remember: false
```

```
38 })
39 const rules = {
40   username: [
41     { required: true, message: '请输入用户名', trigger: 'blur'
42       },
43     { min: 3, max: 20, message: '长度3-20个字符', trigger:
44       'blur' }
45   ],
46   password: [
47     { required: true, message: '请输入密码', trigger: 'blur' },
48     { min: 6, message: '密码至少6位', trigger: 'blur' }
49   ]
50 }
51 const handleSubmit = async () => {
52   await formRef.value.validate()
53   const res = await login(form)
54   userStore.setToken(res.data.token)
55   userStore.setUserInfo(res.data.userInfo)
56   router.push('/')
57 }
58 </script>
```

3.5.2 Pinia 用户状态管理

使用 Pinia 管理用户登录状态，包括 token 存储、用户信息、登录登出操作。通过 localStorage 持久化存储 token，实现刷新页面后保持登录状态。

Listing 12: Pinia 用户状态管理代码

```
1 // store/user.js
2 import { defineStore } from 'pinia'
3 import { ref, computed } from 'vue'
4
5 export const useUserStore = defineStore('user', () => {
6   const token = ref(localStorage.getItem('token') || '')
7   const userInfo = ref(null)
8   const isLoggedIn = computed(() => !!token.value)
9   const isAdmin = computed(() => userInfo.value?.role ===
10     'ADMIN')
11
12   const setToken = (newToken) => {
13     token.value = newToken
```

```
13     localStorage.setItem('token', newToken)
14   }
15   const setUserInfo = (info) => { userInfo.value = info }
16   const logout = () => {
17     token.value = ''
18     userInfo.value = null
19     localStorage.removeItem('token')
20   }
21   return {
22     token, userInfo, isLoggedIn, isAdmin,
23     setToken, setUserInfo, logout
24   }
25 })
```

3.5.3 Axios 请求拦截器

在 Axios 请求拦截器中统一添加 JWT Token 到请求头，在响应拦截器中处理 401 未授权错误。

Listing 13: Axios 封装代码

```
1 // utils/request.js - Axios 封装
2 import axios from 'axios'
3 import { useUserStore } from '@store/user'
4
5 const request = axios.create({
6   baseURL: import.meta.env.VITE_API_BASE_URL,
7   timeout: 10000
8 })
9
10 request.interceptors.request.use((config) => {
11   const userStore = useUserStore()
12   if (userStore.token) {
13     config.headers.Authorization = `Bearer ${userStore.token}`
14   }
15   return config
16 })
17
18 request.interceptors.response.use(
19   (response) => {
20     const res = response.data
21     if (res.code !== 200) {
```

```
22     ElMessage.error(res.message || '请求失败')
23     if (res.code === 401) {
24         const userStore = useUserStore()
25         userStore.logout()
26         window.location.href = '/login'
27     }
28     return Promise.reject(new Error(res.message))
29 }
30 return res
31 },
32 (error) => {
33     ElMessage.error(error.message || '网络错误')
34     return Promise.reject(error)
35 }
36 )
37
38 export default request
```

3.6 后台管理系统设计

后台管理系统是博客的核心运营平台，采用 Element Plus 的 Container 布局容器，左侧为导航菜单，右侧为内容区域。

3.6.1 后台布局组件

BackendLayout.vue 定义后台管理系统的整体布局。使用 el-container 实现经典的侧边栏 + 头部 + 主体的三段式布局。

Listing 14: 后台布局组件代码

```
1 <template>
2   <el-container class="backend-layout">
3     <el-aside width="200px" class="sidebar">
4       <div class="logo-area"><span>博客后台管理</span></div>
5       <el-menu :default-active="activeMenu" router
6         background-color="#304156" text-color="#bfc9d9"
7         active-text-color="#409EFF">
8         <el-menu-item v-for="route in backendRoutes"
9           :key="route.path" :index="route.path"
10          v-show="!route.hidden">
11           <el-icon><component :is="route.meta?.icon"
12             /></el-icon>
```

```
12     <span>{{ route.meta?.title }}</span>
13   </el-menu-item>
14 </el-menu>
15 </el-aside>
16 <el-container>
17   <el-header class="header">
18     <breadcrumb />
19     <el-dropdown>
20       <span>{{ userStore.userInfo?.username }}</span>
21       <template #dropdown>
22         <el-dropdown-menu>
23           <el-dropdown-item
24             @click="goProfile">个人设置</el-dropdown-item>
25           <el-dropdown-item divided
26             @click="logout">退出登录</el-dropdown-item>
27         </el-dropdown-menu>
28       </template>
29     </el-dropdown>
30   </el-header>
31   <el-main>
32     <router-view v-slot="{ Component }">
33       <transition name="fade" mode="out-in">
34         <component :is="Component" />
35       </transition>
36     </router-view>
37   </el-main>
38 </el-container>
</el-container>
</template>
```

3.6.2 仪表盘设计

仪表盘是后台管理的首页，使用 ECharts 图表展示博客的核心数据统计。

Listing 15: 仪表盘组件代码

```
1 <template>
2   <div class="dashboard">
3     <el-row :gutter="20">
4       <el-col :span="6" v-for="stat in stats"
5         :key="stat.title">
6         <el-card class="stat-card">
```



```
6         <div class="stat-icon" :style="{ background:
          stat.color }">
7         <el-icon :size="40"><component :is="stat.icon"
          /></el-icon>
8       </div>
9       <div class="stat-info">
10        <p>{{ stat.title }}</p>
11        <p class="value">{{ stat.value }}</p>
12      </div>
13    </el-card>
14  </el-col>
15</el-row>
16<el-row :gutter="20">
17  <el-col :span="16">
18    <el-card>
19      <template #header>近30天文章发布趋势</template>
20      <div ref="trendChart" style="height: 350px"></div>
21    </el-card>
22  </el-col>
23  <el-col :span="8">
24    <el-card>
25      <template #header>文章分类分布</template>
26      <div ref="pieChart" style="height: 350px"></div>
27    </el-card>
28  </el-col>
29</el-row>
30</div>
31</template>
32
33<script setup>
34import { ref, onMounted } from 'vue'
35import * as echarts from 'echarts'
36
37const stats = ref([
38  { title: '文章总数', value: 0, icon: 'Document', color:
    '#409EFF' },
39  { title: '用户总数', value: 0, icon: 'User', color:
    '#67C23A' },
40  { title: '评论总数', value: 0, icon: 'ChatDotRound', color:
    '#E6A23C' },
41  { title: '今日访问', value: 0, icon: 'View', color:
```

```
        '#F56C6C' }
42  ])
43
44  const trendChart = ref()
45  const pieChart = ref()
46
47  onMounted(async () => {
48    const res = await getDashboardStats()
49    const data = res.data
50    stats.value[0].value = data.articleCount
51    stats.value[1].value = data.userCount
52    stats.value[2].value = data.commentCount
53    stats.value[3].value = data.todayViews
54
55    const trend = echarts.init(trendChart.value)
56    trend.setOption({
57      tooltip: { trigger: 'axis' },
58      xAxis: { type: 'category', data: data.trendDates },
59      yAxis: { type: 'value' },
60      series: [{
61        data: data.trendValues, type: 'line', smooth: true,
62        areaStyle: {
63          color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [
64            { offset: 0, color: 'rgba(64,158,255,0.3)' },
65            { offset: 1, color: 'rgba(64,158,255,0.05)' }
66          ])
67        }
68      }]
69    })
70
71    const pie = echarts.init(pieChart.value)
72    pie.setOption({
73      tooltip: { trigger: 'item' },
74      series: [{
75        type: 'pie', radius: ['40%', '70%'],
76        itemStyle: { borderRadius: 10, borderColor: '#fff',
77          borderWidth: 2 },
78        data: data.categoryDistribution
79      }]
80    })
  })
```

```
81 </script>
```

3.6.3 文章管理

文章管理页面提供文章的增删改查功能。列表页使用 el-table 展示文章数据，支持分页、排序、筛选。新增/编辑页使用 mavon-editor 组件提供 Markdown 编辑功能。

Listing 16: 文章编辑页代码

```
1 <template>
2   <div class="article-edit">
3     <el-form :model="form" :rules="rules" ref="formRef"
4       label-width="80px">
5       <el-form-item label="标题" prop="title">
6         <el-input v-model="form.title"
7           placeholder="请输入文章标题" />
8       </el-form-item>
9       <el-form-item label="分类" prop="categoryId">
10        <el-select v-model="form.categoryId"
11          placeholder="选择分类">
12          <el-option v-for="cat in categories" :key="cat.id"
13            :label="cat.name" :value="cat.id" />
14        </el-select>
15      </el-form-item>
16      <el-form-item label="标签">
17        <el-select-v2 v-model="form.tagIds"
18          :options="tagOptions"
19          placeholder="选择标签" multiple clearable />
20      </el-form-item>
21      <el-form-item label="封面">
22        <el-upload class="cover-uploader" :action="uploadUrl"
23          :headers="headers" :show-file-list="false"
24          :on-success="handleUploadSuccess">
25          
27          <el-icon v-else class="uploader-icon"><Plus
28            /></el-icon>
29        </el-upload>
30      </el-form-item>
31      <el-form-item label="内容" prop="content">
32        <mavon-editor v-model="form.content"
33          :toolbars="toolbars">
```

```
27         @imgAdd="handleEditorImgAdd" style="height: 500px" />
28     </el-form-item>
29     <el-form-item>
30         <el-button type="primary" @click="submitForm"
31             :loading="saving">
32             {{ isEdit ? '保存修改' : '发布文章' }}</el-button>
33         <el-button @click="$router.back()">取消</el-button>
34     </el-form-item>
35 </el-form>
36 </div>
</template>
```

3.7 响应式布局设计

本项目采用多种技术实现响应式布局，确保在不同设备上都有良好的浏览体验。

3.7.1 Flexbox 弹性布局

Flexbox 用于一维布局场景，如头部导航、表单元素排列、卡片内部结构等。

3.7.2 CSS Grid 网格布局

CSS Grid 用于二维布局场景，如文章列表的网格排列。使用 `auto-fill` 和 `minmax` 函数实现自适应列数。

Listing 17: 响应式布局 CSS 代码

```
1  /* 响应式网格布局 */
2  .article-grid {
3      display: grid;
4      grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
5      gap: 24px;
6  }
7
8  @media screen and (max-width: 768px) {
9      .article-grid { grid-template-columns: 1fr; }
10     .sidebar { display: none; }
11     .main-content { width: 100%; }
12 }
```

3.7.3 Element Plus 响应式组件

Element Plus 的栅格系统 (el-row/el-col) 内置了响应式支持, 通过 xs、sm、md、lg、xl 属性设置不同屏幕尺寸下的列宽。

3.8 底部设计

底部 (Footer) 区域位于页面最下方, 包含版权信息、友情链接、联系方式等内容。

3.8.1 底部布局

底部采用深色背景, 三栏布局。左侧为博客简介和 Logo; 中间为友情链接列表, 以标签形式展示; 右侧为联系方式和社交媒体链接。

Listing 18: 底部组件代码

```
1 <template>
2   <footer class="app-footer">
3     <div class="footer-content">
4       <div class="footer-section about">
5         <h4>个人博客</h4>
6         <p>记录学习, 分享技术, 探索未知。</p>
7       </div>
8       <div class="footer-section links">
9         <h4>友情链接</h4>
10        <div class="link-tags">
11          <a v-for="link in friendLinks" :key="link.id"
12            :href="link.url" target="_blank" class="link-tag">
13            {{ link.name }}</a>
14        </div>
15      </div>
16      <div class="footer-section contact">
17        <h4>联系方式</h4>
18        <p><el-icon><Message /></el-icon>
19          contact@example.com</p>
20        <p><el-icon><Location /></el-icon> 北京市</p>
21      </div>
22      <div class="copyright">
23        <p>&copy; 2026 个人博客 版权所有</p>
24      </div>
25    </footer>
```

26 </template>

4 总体页面效果

本章展示博客系统各主要页面的整体效果。由于页面截图无法直接在文档中展示，以下通过文字描述各页面的布局和元素。

4.1 前台页面效果

4.1.1 首页效果

首页整体效果大气现代，深色调背景搭配亮蓝色强调色，营造专业的技术博客氛围。顶部为固定导航栏，左侧为博客 Logo 和名称，中间为水平导航菜单（包含二级下拉菜单），右侧为搜索框和用户入口。

英雄区域占满首屏高度，左侧为打字机效果的欢迎标题和副标题，右侧为推荐文章轮播图。底部为 SVG 波浪动画，自然过渡到内容区域。内容区域展示最新文章卡片网格，每张卡片包含封面图、标题、摘要、作者信息和标签。底部分页组件支持页码跳转和每页数量选择。

4.1.2 文章列表页效果

文章列表页采用左右两栏布局。左侧为主内容区，顶部为面包屑导航和排序选项（最新、最热、最多评论），下方为文章卡片列表。与首页不同的是，列表页卡片采用水平布局（图片在左，内容在右），信息密度更高。右侧为侧边栏，包含搜索框、分类列表、标签云、热门文章推荐。

4.1.3 文章详情页效果

文章详情页同样为左右两栏布局。左侧主内容区从上到下依次为：文章标题、作者和发布元信息、Markdown 渲染的正文内容（包含代码高亮块）、文章标签、点赞/收藏按钮、上一篇/下一篇导航、评论区。右侧为粘性侧边栏，包含作者信息卡片、目录导航（自动高亮当前阅读位置）、相关文章推荐。

4.1.4 分类/标签页效果

分类页面展示所有文章分类的卡片网格，每个分类卡片显示分类名称、描述和文章数量。点击分类进入该分类下的文章列表。标签页以标签云的形式展示所有标签，标签大小根据文章数量动态调整。

4.1.5 用户认证页面效果

登录/注册页面背景为深色粒子动画，中央为半透明的登录卡片。卡片包含标题、输入表单（带图标前缀）、验证码图片、提交按钮。表单验证实时反馈，输入错误时显示提示信息。

4.1.6 个人中心页面效果

个人中心采用卡片式布局，包含以下功能模块：个人资料卡片、资料编辑表单、我的文章列表、我的收藏列表、我的评论记录、我的点赞记录。

4.2 后台管理页面效果

4.2.1 仪表盘效果

后台仪表盘为数据概览页面。顶部为四张统计卡片，分别以不同颜色展示文章数、用户数、评论数、今日访问量。下方左侧为近 30 天文章发布趋势折线图，右侧为文章分类分布饼图。图表采用 ECharts 渲染，支持交互提示。

4.2.2 文章管理效果

文章管理页面上方为搜索栏和筛选条件，以及“写文章”按钮。下方为 el-table 表格，列包括 ID、标题、分类、作者、状态、创建时间、操作。操作列包含编辑、删除、预览按钮。

4.2.3 文章编辑页面效果

文章编辑页面为表单布局，包含标题输入框、分类选择器、标签多选器、封面上传组件和 Markdown 编辑器。Markdown 编辑器占据主要区域，高度 500px。

4.2.4 用户/评论管理效果

用户管理页面以表格形式展示注册用户列表，支持按用户名搜索、按角色筛选。评论管理页面展示所有评论列表，支持对评论进行审核（通过/拒绝）和删除操作。

5 心得体会

通过本次 Web 前端开发课程项目的实践，我获得了丰富的开发经验和技能成长。以下是我在项目开发过程中的主要心得体会。

5.1 技术能力提升

通过本次项目，我深入掌握了 Vue 3 框架的核心概念和最佳实践。特别是 Composition API 的使用让我对 Vue 组件的逻辑组织有了全新的认识。相比于 Options API，Composition API 通过 setup 函数将相关逻辑聚合在一起，使得代码更加清晰易维护。我学会了如何使用 ref 和 reactive 创建响应式数据，如何使用 computed 计算属性，如何使用 watch 监听数据变化，以及如何使用 provide/inject 进行跨层级依赖注入。

路由管理方面，我深入学习了 Vue Router 4 的嵌套路由、动态路由、路由守卫等高级特性。通过路由守卫实现了基于角色的权限控制，确保未登录用户无法访问后台管理页面。路由懒加载的使用也显著提升了首屏加载速度。

状态管理方面，Pinia 的简洁 API 设计让状态管理变得轻松愉快。我学会了如何合理划分 Store 模块，如何将 Pinia 与 localStorage 结合实现状态持久化，以及如何在组件外部使用 Store。

5.2 工程化意识培养

本项目让我深刻认识到前端工程化的重要性。通过使用 Vue CLI 搭建项目，我了解了 Webpack 的构建流程、Babel 的代码转译、ESLint 的代码规范检查等工程化工具的作用。合理配置路径别名（@ 指向 src 目录）让模块导入更加简洁清晰。

代码规范方面，我养成了良好的编码习惯：组件命名采用大驼峰（PascalCase），组合式函数使用 use 前缀，API 接口按模块组织，工具函数独立封装。注释规范方面，关键逻辑添加清晰的注释，复杂算法解释实现思路。

性能优化方面，我学习了组件懒加载、图片懒加载、虚拟滚动、防抖节流等优化技巧。特别是在处理大量评论数据时，递归组件的合理使用避免了页面卡顿。

5.3 UI/UX 设计感悟

在项目开发过程中，我深刻体会到用户体验的重要性。一个好的产品不仅要功能完善，更要让用户用得舒服。我学习了响应式设计的核心原则，确保博客在手机、平板、电脑等不同设备上都能正常显示。

动画效果的合理运用可以提升用户体验。本项目中的打字机效果、粒子背景、波浪动画、卡片悬浮效果等，都是通过 CSS 动画和 JavaScript 配合实现的。但我也认识到动画要适度，过多的动画反而会影响用户体验。

色彩搭配上，我研究了深色主题的设计方法。通过合理的对比度控制、强调色的点缀，营造出既美观又护眼的深色界面。Element Plus 的主题定制功能让组件库与整体风格完美融合。

5.4 问题解决能力

项目开发过程中遇到了许多技术难题，每解决一个问题都是一次成长。例如：Markdown 编辑器图片上传的处理、多级评论的递归渲染、ECharts 图表的响应式适配、JWT Token 过期刷新机制等。通过查阅官方文档、搜索技术博客、阅读源码等方式，我逐步建立了解决问题的能力和信心。

前后端联调是一个考验耐心的过程。接口数据格式不一致、跨域问题、认证失败等问题都需要前后端协同解决。通过这个过程，我学会了如何使用浏览器开发者工具调试网络请求，如何阅读后端接口文档，如何与后端开发有效沟通。

5.5 总结与展望

本次课程项目是一次完整的 Web 前端开发实践，从需求分析、技术选型、UI 设计到编码实现、测试调试、部署上线，完整经历了软件开发的整个生命周期。通过这个项目，我不仅巩固了课堂所学的 Vue3 知识，还拓展学习了 Element Plus、ECharts、Markdown 编辑器等第三方库的使用。

展望未来，我希望继续深耕前端技术领域。下一步学习计划包括：深入 TypeScript 类型系统，提升代码质量；学习前端性能优化的高级技巧；了解服务端渲染（SSR）和静态站点生成（SSG）；探索微前端架构等前沿技术。本项目为我的前端技术之路奠定了坚实的基础，我会继续保持学习的热情，不断提升自己的技术水平。